

# SwishOS — Stage 2: Fleet Soiling & Cleaning Advisor

---

**Role:** Full-Stack Developer, Agentic Systems

**Time:** 4 hours for the build, roughly 30 minutes for the slides.

**Live session:** 45 minutes. You present for 15, then we discuss your decisions and code.

---

## Scope

This is the technical round. It covers both engineering and product judgment.

We are assessing your decisions: what you built, what you chose not to build, in what order, and what each choice cost.

We have not specified an implementation. There is no list of endpoints, patterns, or file layout below — you get the problem, the constraints and the data, and decide what the system should be.

We expect you to use AI. We have no preference for any particular approach, but we need the reasoning behind what you build.

An incomplete submission with clear reasoning scores higher than a complete one without.

---

## Time

Four hours for the build, roughly 30 minutes for the slides.

Scope isn't what we're scoring. Every decision has to be justified in your log and defended in the session, so anything extra you build is something extra to explain.

You're done when the system runs, the decision log covers what you chose, and the slides are ready. If you're past five hours and still building, write down what you would have done next and stop there.

---

## Background

Soiling — dust, pollen, bird droppings, industrial fallout on panels — is the largest controllable loss in a solar plant. It builds gradually, resets when it rains, and resets fully when a crew cleans.

Cleaning costs money, water and truck time. Clean too early and you spend more than you recover. Clean too late and the energy is already lost. Rain tomorrow makes today's cleaning worthless.

Swish monitors 100 plants today and will monitor 200 within six months. Cleaning schedules are still decided in a spreadsheet, plant by plant, which stopped working some time ago: a few familiar sites get attention and the rest run on a fixed three-week rota regardless of condition.

Two people depend on the outcome:

- **An asset manager** with 200 plants. She approves the spend.
  - **A crew lead** with a truck and a water tank. He does the work.
- 

## The data

`seed_data.py` generates the dataset. Run it with the default seed.

```
python seed_data.py                                # 12 plants × 120 days
python seed_data.py --plants 200 --days 540 --out data_scale # a larger fleet, if useful
```

Standard library only. The default run is about 7 MB; the larger one is about 530 MB and takes a few minutes. Don't commit the output.

`daily.csv` — one row per plant per day. Start here.

| Column                           | Meaning                                                                                                                                  |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>energy_kwh</code>          | Actual generation for the day, in kWh.                                                                                                   |
| <code>expected_energy_kwh</code> | What the plant should have generated given that day's weather, in kWh. Already corrected for irradiance and temperature.                 |
| <code>pr</code>                  | Performance ratio: <code>energy_kwh / expected_energy_kwh</code> . A ratio, not a percentage — 1.0 is ideal, 0.86 means 14% below ideal. |
| <code>soiling_loss_pct</code>    | Output lost to dirt, in percent. <code>3.9</code> means 3.9%. Blank where there is no clean baseline to compare against yet.             |

`plants.csv` — one row per plant.

| Column                             | Meaning                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------|
| <code>capacity_mw</code>           | Nameplate capacity, in MW.                                                        |
| <code>tariff_per_kwh</code>        | USD earned per kWh generated.                                                     |
| <code>cleaning_cost_usd</code>     | USD to clean the whole plant once.                                                |
| <code>days_until_next_reset</code> | Typical days before rainfall washes this plant clean by itself. Varies by region. |
| <code>commissioned_on</code>       | Date the plant came online, <code>YYYY-MM-DD</code> .                             |

`events.csv` — one row per plant per day. `rain_mm` is that day's rainfall in millimetres; roughly 8mm or more washes the panels clean. `cleaned` is `1` if a crew cleaned that day, `0` otherwise.

`crews.csv` — `mw_per_day` is how much plant a crew can clean in a day, in MW. `day_rate_usd` is what the crew costs per day.

`readings/plant_<id>.csv` — the raw 15-minute telemetry `daily.csv` is derived from. You shouldn't need it.

### Domain formulas

`pr` and `soiling_loss_pct` are already computed in `daily.csv`. The one you need:

```
recoverable_usd = (expected_energy_kwh * soiling_loss_pct / 100)
                  * tariff_per_kwh
                  * days_until_next_reset
                  - cleaning_cost_usd
```

`days_until_next_reset` comes from `plants.csv`. Forecasting rainfall yourself is out of scope — use the column.

A starting point. Improve on it if you want to, and say why.

## What to build

Enough of the system that the asset manager could decide where to send a crew tomorrow, and that decision could be carried out.

That means something has to:

- turn the readings above into a view of where cleaning is worth doing
- let a human act on one of those recommendations
- use an LLM somewhere you can justify, and not where you can't

How you decompose that is your call.

### Frontend and backend

Both are required. There must be an interface a person can use, talking to a backend you wrote, and both must run locally from a clean clone.

We use React/Next.js with TypeScript, and Python with FastAPI. Use that if you're comfortable in it. If you work faster in something else, use that and note it in the README — we care that it runs and that you can defend it, not which framework it's in.

It should run without hosted services or accounts. If your LLM step needs an API key, make sure the rest of the system still runs without one.

Which screen, for which of the two people, what it shows and how it's structured are all open. Four hours covers everything, so how much of it the interface deserves is part of what you're deciding.

### Constraints

- 200 plants within six months, five years of retention, and a fleet-wide view that loads for around 120 people shortly after 8am.
- Acting on a recommendation dispatches a truck and spends roughly what it expects to recover.
- The dataset contains deliberate data-quality defects, as real telemetry does.

---

## Deliverable 1 — Decision log

DECISIONS.md, covering each decision you made:

| Field     | What to write                                   |
|-----------|-------------------------------------------------|
| Decision  | What you did.                                   |
| Rejected  | What else you considered, and why it lost.      |
| Cost      | What this choice makes worse, harder or slower. |
| Falsifier | What would make you change your mind.           |

Also list the assumptions you made and did not verify — about the data, the domain, or how the system would be used. The ones worth listing are those that would change what you built if they turned out to be wrong.

We read this before the code.

---

## Deliverable 2 — Code

Frontend and backend, running from a clean clone using the commands in your README.

We are not scoring lines, files or test coverage. We are scoring whether the system matches what you described and whether you can defend it.

---

## Deliverable 3 — Presentation

Five slides, presented in 15 minutes at the start of the live session, including two minutes of the system running. Please keep to the time.

1. What you built, and what you chose not to build. Demo here.
2. The decision you are least sure about.
3. What breaks first.
4. What you would build next, and in what order.
5. What you would change about this brief.

---

## Submitting

A git repo or zip containing:

```
README.md      how to run both halves from a clean clone
DECISIONS.md   the decision log
slides.pdf     or .pptx / .key
<backend>/     API and data layer
<frontend>/    interface
```

Organise it however you like. We will clone and run it — if setup takes more than a few commands, say so in the README.

---

## Live session

45 minutes. Your presentation opens it. We will then select files and ask about the decisions in them.

Questions by email during the exercise are welcome.